



**Facultad de
Ingeniería**

Universidad Católica de la Santísima Concepción

Programación Concurrente

Hilos

Sistemas Operativos (2024_1 - IN1077C)

Laboratorio

Ayudante: Derqui Sanhueza Balboa



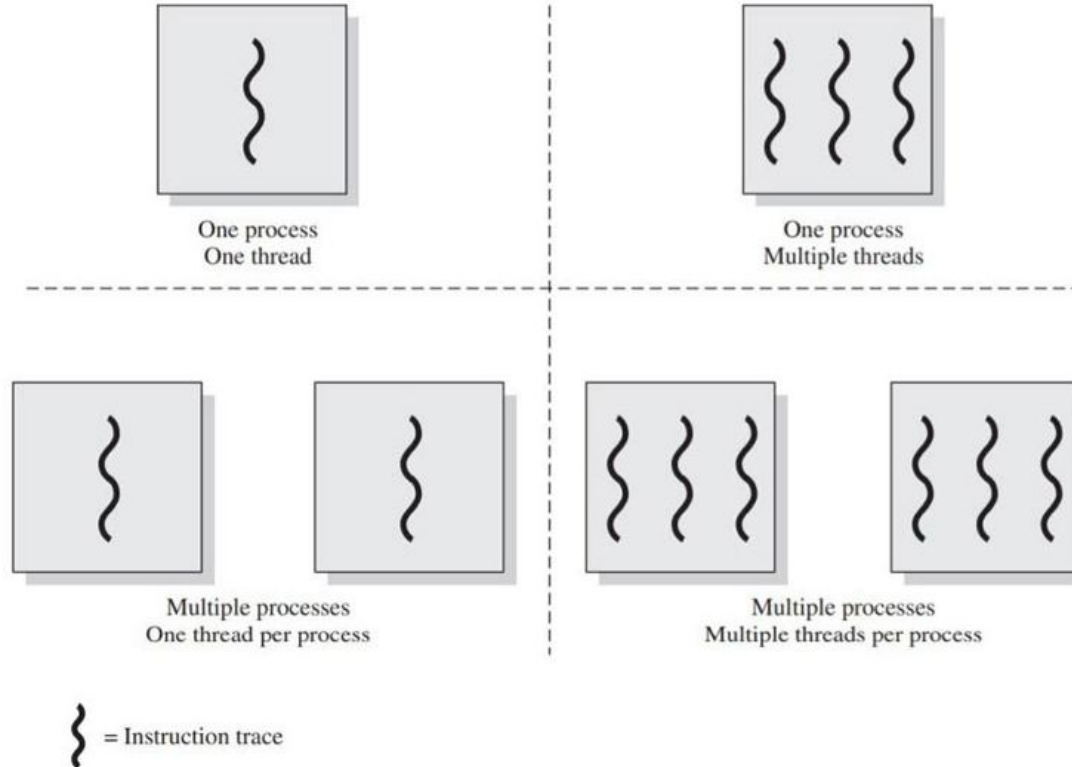
¿Qué es un Hilo?

Secuencia de instrucciones dentro de un programa que puede ejecutarse concurrentemente con otros hilos dentro del mismo proceso.

Para que un proceso se ejecute, debe tener al menos un hilo.

Ejemplo: chrome.exe es el proceso principal del navegador Google Chrome, los navegadores modernos como este utilizan hilos para delegar diferentes tareas como: Renderizar páginas web, reproducir contenido multimedia, gestionar operaciones de red, etc.

¿Qué es un Hilo?





Biblioteca pthread.h

Pthreads es una biblioteca en C/C++ que nos permite trabajar con hilos en sistemas Unix. 'Pthreads' significa 'Posix Threads', y es una implementación del estándar POSIX (Portable Operating System Interface) para hilos.



Uso de pthread.h

#include <pthread.h>

Declaración de los hilos que se utilizarán

pthread_t hilo1, hilo2, ..., hilo(n-ésimo);

Creación de los hilos

pthread_create(*pthread_t *thread, const pthread_attr_t *attr, void *(*start)(void *), void *arg*);



Uso de pthread.h

```
pthread_create(pthread_t *thread, const pthread_attr_t *attr, void *(*start)(void *), void *arg);
```

Dirección de memoria de donde se guardara el hilo

Atributos del hilo que se esta creando

Funcion que ejecutará el hilo

Parámetro que se pasa al iniciar el hilo



Uso de pthread.h

Esperar a que los hilos se terminen de ejecutar

```
pthread_join(pthread_t thread, void **value_ptr);
```

Variable de hilos declarados anteriormente

Dirección de memoria de donde se quiera guardar el resultado de ejecución del hilo



Uso de pthread.h

Para manejar las condiciones de carrera se utilizará el método de exclusión mutua, lo que significa que podemos “bloquear” ciertas partes del código que son críticas para que sean accesibles por un hilo a la vez.

Para esto utilizaremos el método MUTEX de la biblioteca pthread.h



Uso de pthread.h

```
pthread_mutex_t mutex;
```

```
pthread_mutex_init(pthread_mutex_t * mutex, const pthread_mutexattr_t * attr);
```

```
pthread_mutex_lock(pthread_mutex_t * mutex);
```

```
pthread_mutex_unlock(pthread_mutex_t * mutex);
```

```
pthread_mutex_destroy(pthread_mutex_t * mutex);
```